26T1 Week04

Design Principles

Software Design & Architecture

Annelise

agenda

- Cohesion and Coupling
- SOLID Principles
- Law of Demeter
- Composite Pattern
- Factory Pattern

cohesion and coupling

- **High Cohesion**
  - A class should do one thing and do it well
  - All its methods and properties work together toward a single purpose
  - Easy to understand, reuse, and maintain
- **Low Coupling**
  - Classes should depend on abstractions, not concrete details
  - Minimize what one class knows about another
  - Changes in one class shouldn't force changes in others
- Together they create systems that are easy to modify, test, and reason about

- A class should have only one responsibility
- A class should have only one reason to change
- Keeps classes smaller, clearer, and easier to maintain
- Encourages separation of concerns

single responsibility principle

- Software entities should be **open for extension, but closed for modification**
- Existing code should not need to change when new behaviour is added
- New functionality should be added through new classes or implementations
- Achieved using abstraction, interfaces, and polymorphism

open/closed principle

Liskov's Substitution Principle

"Objects of a **superclass should be replaceable** with objects of its subclasses **without breaking** the application or altering expected behavior."

Liskov's Substitution Principle

LSP is violated when:

Preconditions are strengthened,
Postconditions are weakened,
or Invariants are not held

in the subclass

- LSP violations are often caused by inheritance
- Inheritance enables code reuse, but subclasses must inherit all behaviour
  - Sometimes parts of the superclass don't make sense for the subclass, and this can break LSP
- To avoid this
  - Minimise unnecessary inheritance
  - Prefer composition and delegation
  - **Delegation**: pass work to another object that handles it
  - **Composition**: build a class using other classes instead of inheriting from them
- *Guideline: Favour composition over inheritance*
  -

composition over inheritance ✦

- Clients should not be forced to depend on methods they do not use
- **Large interfaces should be split** into smaller, more specific interfaces
- Classes **implement only the methods relevant** to them
- Leads to more flexible and maintainable designs

interface segregation

😿 lacks segregation; Robot has to implement unusable methods

```
interface Worker {
  work();
  eat();
  sleep();
}

class Robot implements Worker {
  work() {...}
  eat() { throw "Can't eat"; } //
  sleep() { throw "Can't sleep"; }
}
```

```
interface Workable {
  work();
}

interface Eatable {
  eat();
}

interface Sleepable {
  sleep();
}

class Human implements Workable, Eatable, Sleepable {...}
class Robot implements Workable {...}
```

😸 each class only implements relevant interfaces

✸ interface segregation

- High-level modules should not depend on low-level modules
- Both should **depend on abstractions** (interfaces or abstract classes)
- Abstractions should not depend on details; **details should depend on abstractions**
- Reduces tight coupling between components
- Makes code easier to change, test, and extend

dependency inversion

```java
class EmailSender {
  void send(String message) { ... }
}

class Notification {
  private EmailSender email = new EmailSender();

  void alert(String text) {
    email.send(text); // Can only send via email
  }
}
// To add SMS, must modify Notification class
```

Bad – high level module (Notification) depends on low-level module (EmailSender).

If we wanted to .alert() by SMS, push or any other message service, we'd have to modify the Notification class for each one of those services.

dependency inversion

```java
interface MessageService {
    void send(String msg);
}
```

```java
class Notification {
    private List<MessageService> services;

    Notification(List<MessageService> services) {
        this.services = services;
    }

    void notifyUser(String msg) {
        for (MessageService service : services) {
            service.send(msg);
        }
    }
}
```

Much better! **Notification depends on an abstraction** (interface), rather than a specific service like EmailSender.

The system is more **extensible**, since we can notifyUser() with email, SMS, push, etc. **without modifying Notification** every time we want a new MessageService.

dependency inversion

1. **S**ingle Responsibility Principle (SRP)
2. **O**pen/Closed Principle (OCP)
3. **L**iskov's Substitution Principle (LSP)
4. **I**nterface Segregation
5. **D**ependency Inversion

✦ ✦ ✦

summary of SOLID

applying SOLID

For each of the following scenarios, pick the **most applicable SOLID design principle** being violated, and discuss how the systems can be refactored to remove the violation.

applying SOLID

For assignment-i, a Student has decided to implement all the movement logic inside of the TrainsController class's simulate() function.

As such, their Train classes did not contain much code except for a few getter and setter functions.

applying SOLID

Chloe has a Cafe class that serves different types of Coffee. **Each type of coffee has a different way** to grindBeans(), heatWater(), brew(), and serve(). This was implemented with a **switch statement** in each method, but when Chloe wanted to introduce a new type of coffee to her system, she found that she had to modify many methods of the Cafe class.

applying SOLID

Nicole's game has different character with certain abilities, such as:

- Sorcerer, who can only cast spells
- Knight, who can melee attack
- Herald, who can cast spells and melee attack

She has an CharacterAbility interface that contains the methods castSpell() and meleeAttack(). However she has found that this implementation meant that some of the character subclasses had to override these methods to do nothing.

applying SOLID

Sebastian is writing a Switch class which directly controls a LightBulb class. However, when he wanted to **update his implementation** to support an InfraredBulb, he had to make a **lot of changes to the logic in the Switch** class to make it work with the new type of bulb.

Law of Demeter

A method m of an object O should only call methods from:

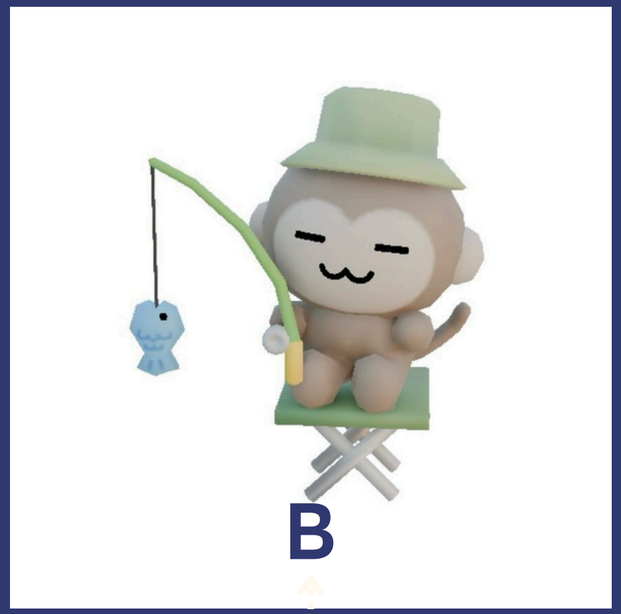
- The object O itself
- Objects that are member variables of o
- Objects passed in as parameters to m
- Objects instantiated within m

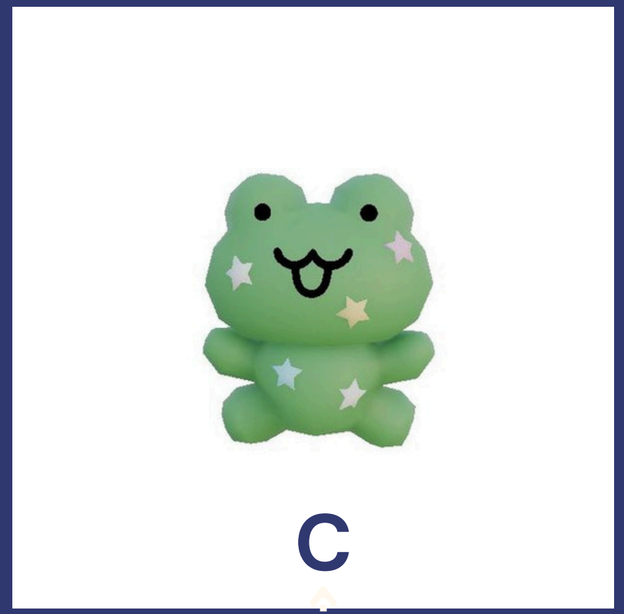
Avoid method chaining (e.g. something like obj.getX().getY().doSomething())

Law of Demeter

B is a “friend” of A

C is a “stranger” to A





messages from
A to B are okay!

messages from A to C
are discouraged

example

```java
interface FlyBehaviour {
    void fly(int height);
}
```

```java
class Bird {
    private FlyBehaviour flyBehaviour;

    public Bird(FlyBehaviour flyBehaviour) {
        this.flyBehaviour = flyBehaviour;
    }

    public void fly(int height) {
        flyBehaviour.fly(height); // delegation
    }
}
```

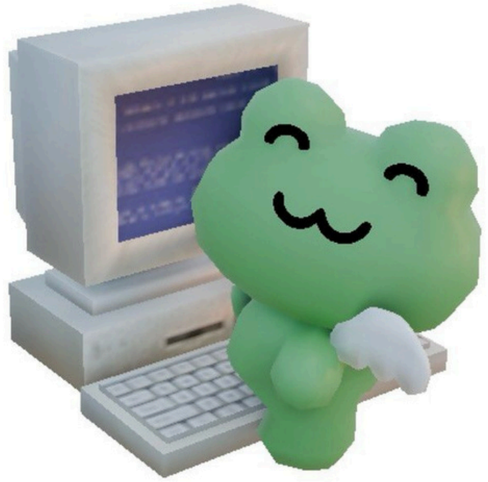
```java
Bird eagle = new Bird(new NormalFly());
Bird penguin = new Bird(new NoFly());

eagle.fly(10);
penguin.fly(0);
```

This is what it could look like to fix the Bird/Penguin problem from last week.

Classes like NoFly and NormalFly would **implement the FlyBehaviour interface**. Then, we make our **Bird class compose of a FlyBehaviour**.

By delegating the flying logic, we can now call .fly() safely on flying/flightless birds!

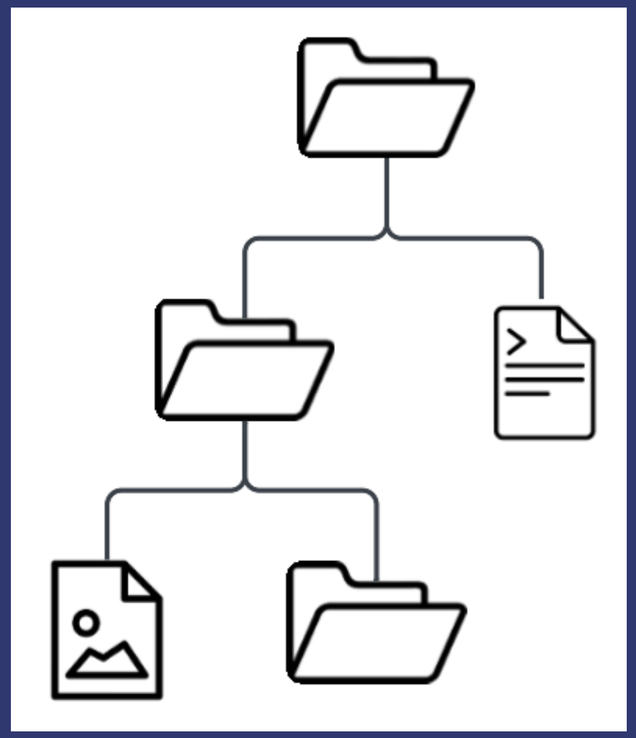


code review

Training

composite pattern

Structural



Suppose we have

- A File can store data like text, images etc
- A Directory can store both files and other directories

Together files and directories form a hierarchy that makes up a FileSystem.

Code becomes messy and harder to maintain because files and directories cannot be handled uniformly.

the problem

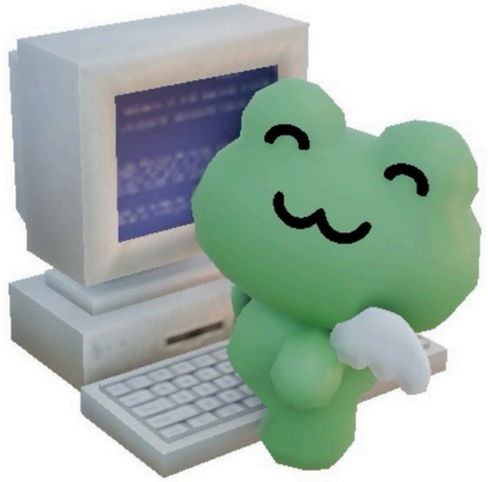
What is it?

- A *structural design pattern* that allows a group of objects to be treated like single instances of the object
- Single instance: File
- Group of objects: Directory

When to use?

- Model a family of classes in a tree-like hierarchy to represent the part-whole relationship

composite pattern



code demo!

Calculator

Use the **composite pattern** to design a simple calculator that can be used to evaluate an arithmetic expression

Your calculator should be able to:

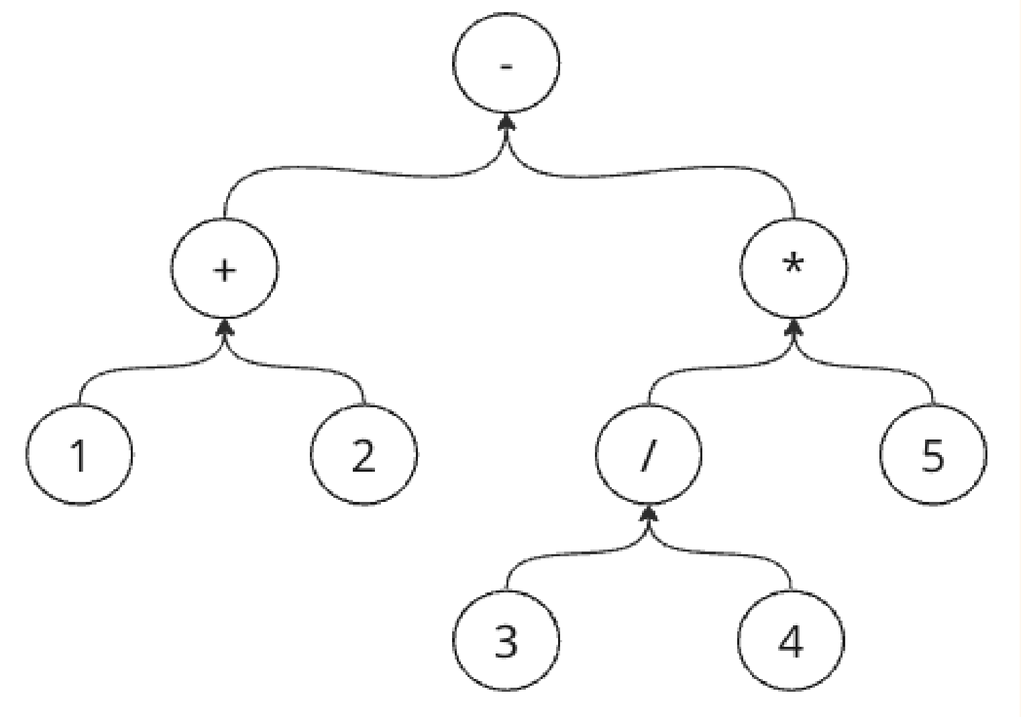
- Add two expressions
- Subtract two expressions
- Multiply two expressions
- Divide two expressions

calculator ✦

- First, we break the expressions into binary-tree like structure. We can use the composite pattern to construct the expressions.
- Since numbers are just numbers, there's no need to compose anything, so they are our Leaf.

```
Expression exp1 = 42;
```

calculator ✦



- Numeric operators are a bit different as they are Composites.
- However, operators always have an expression for the left and right hand sides (two children). These can either be other Operators or Numbers, because they are both Expressions.

```
Expression exp2 = ((1 + 2) - ((3 / 4) * 5));
```

calculator ✦

factory pattern

Creational

Suppose we have an **ImageViewer** application that lets users delete photos from their mobile device.

We need a **separate delete Button** for each OS (IOS and Android).

BUT we want to **avoid manually checking** cases in a constructor (e.g. if OS == IOS then create <IOSButton> else create <AndroidButton>)

the problem

What is it?

- *Creational design pattern* that lets you create objects without specifying the exact class of object to be created

- *Reduces coupling* between client code and the specific classes being instantiated. Good for **abstraction**, as users simply call a method from the factory
- Construction of every class in the family is handled in the same place

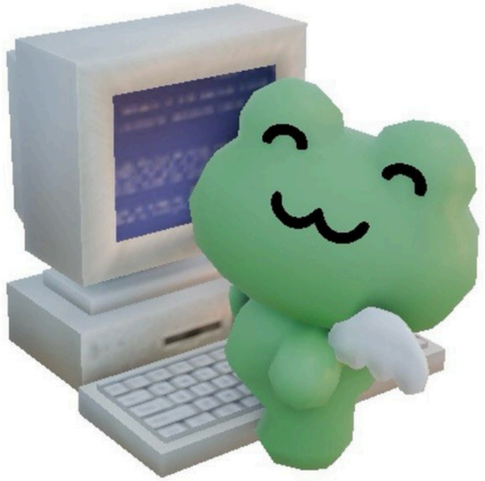
factory pattern

When to use?

Use it when...

- Exact type of object needed is decided at runtime
- You want to separate object creation from the code that uses the objects
- A class may need to create different types of related objects
- You want to avoid tightly coupling code to specific classes

factory pattern



code demo!

Thrones

lab time!

SitarME306